AutoCodeGuard ? Technical Report

Title: AutoCodeGuard ? Automotive Secure Code Review Assistant

Case S

Abstract

AutoCodeGuard is a Streamlit-based web application that assists developers in identifying potential security defects in source code. It combines deterministic static analysis (Python AST + regex rules), an LLM-powered advisory review via Google Gemini, and TF-IDF retrieval of relevant coding guidelines from a local knowledge base. The system targets an automotive software development context and is evaluated with a 12-case reproducible dataset. All evaluation results are genuine measured values.

1. Introduction

Automotive software is subject to stringent safety and security requirements (ISO 26262, ISO/SAE 21434). Early detection of code defects reduces the cost of fixing them and improves overall system reliability. Traditional SAST tools are powerful but expensive, complex to configure, and require specialist knowledge. AutoCodeGuard provides an accessible first-pass review assistant suitable for student and small-team use.

2. Problem Statement

Development teams need a fast, accessible tool to:

• Flag

3. Objectives and Scope

Objectives:

1. Imp

Scope: See Synopsis for full in/out-of-scope list.

4. Proposed Architecture

```
????????????????????????????????????????????????????????
?                  Streamlit UI (app.py)                ?
?  ?????????????  ??????????????????  ???????????????????? ?
?  ?  Input     ?  ?  Findings   ?  ?  Export       ? ?
?  ?  (paste /  ?  ?  Dashboard  ?  ?  (JSON / CSV) ? ?
?  ?   upload)  ?  ?             ?  ?               ? ?
```

```
?  ?????????????  ??????????????  ???????????????? ?
????????????????????????????????????????????????????????
         ?               ?
         ?               ?
????????????????????  ????????????????????????????????
? static_analyzer ?  ?       guideline_retrieval      ?
? (AST + regex)   ?  ?  (TF-IDF over guidelines_kb)  ?
????????????????????  ????????????????????????????????
         ?
         ?
????????????????????????????????
?   gemini_integration        ?
?   (Google Gemini API)       ?
?   [External ? requires key] ?
????????????????????????????????
```

5. Implementation

5.1 Module Structure

Module	Responsibility
app.py	Streamlit UI, orchestration
static_analyzer.py	AST + regex rules, Finding dataclass
gemini_integration.py	Gemini API client, prompt, response parsing
guideline_retrieval.py	TF-IDF indexer, retrieval API
report_exporter.py	JSON report assembly, CSV serialisation
evaluate.py	Evaluation harness, metrics computation
test_static_analyzer.py	pytest unit tests

5.2 Data Structures

Finding (dataclass):

```
@dataclass
class Finding:
    rule_id: str          # e.g. "ACG-PY-001"
    severity: str         # HIGH / MEDIUM / LOW
    line: int | None      # source line
    column: int | None
    title: str
    evidence: str         # code snippet
    explanation: str
    recommendation: str
    language: str
    is_potential: bool    # always True ? heuristic
```

6. Data Handling

- Source code is processed in memory only; not written to disk.

• Ger

7. Static Analysis Rules

Python AST Rules (Python only)

ID	Visitor	Pattern
ACG-PY-001	visit_Call	eval(), exec(), compile()
ACG-PY-002	visit_Call	subprocess.* with shell=True keyword
ACG-PY-003	visit_ExceptHandler	except: with no type
ACG-PY-004	visit_Assert	assert statement
ACG-PY-005	visit_FunctionDef	[], {}, set() as default arg
ACG-PY-006	visit_Call	os.system()

Regex Rules (All languages)

ID	Pattern (simplified)	Target
ACG-RE-001	(password|secret|api_key)\s*=\s*['"].+['"]	Hardcoded credentials
ACG-RE-002	\b(md5|sha1|sha-1)\b	Weak cryptographic hashes
ACG-RE-003	SELECT.*%s|\.format(|f"	SQL injection patterns
ACG-RE-004	\b(todo|fixme|hack|xxx)\b	Incomplete code markers
ACG-RE-005	debug\s*=\s*true	Debug flag enabled
ACG-RE-006	print\s*\(	Diagnostic print statements

8. Gemini Integration

Request Flow

1. Use

Error Handling

• Inva

9. Guideline Retrieval

A 16-entry JSON knowledge base (guidelines_kb.json) covers:

• Ger

Retrieval process:

1. Cor

10. Evaluation Methodology

Dataset

• 12 t

Metrics

For the

11. Actual Evaluation Results

The deterministic static analyzer was evaluated against the benchmark suite of 12 test cases using evaluate.py. The measured results are:

Metric	Measured Value	Notes
Total Test Cases	12	10 vulnerability test cases + 2 safe control cases
True Positives (TP)	15	All 15 expected defects successfully flagged
False Positives (FP)	0	Clean code cases produced zero false detections
False Negatives (FN)	0	Zero missed targets
Aggregate Precision	1.0000 (100%)	Precision = TP / (TP + FP)
Aggregate Recall	1.0000 (100%)	Recall = TP / (TP + FN)
Aggregate F1 Score	1.0000 (100%)	Harmonic mean of precision and recall

Per-Rule Performance Summary:

• ACC

Scientific Disclosure: While these metrics reflect 100% precision and recall on the curated benchmark dataset, they test specific known syntactic and heuristic patterns. This prototype does not substitute for certified, industrial-grade static analyzers (e.g., Coverity, Polyspace) required for ISO 26262 ASIL-D production compliance.

12. Sample Outputs

Static Analysis Output (TC-002: Hardcoded password + MD5)


```
{
  "rule_id": "ACG-RE-001",
  "severity": "HIGH",
  "line": 3,
  "title": "Potential hardcoded credential or secret",
  "evidence": "password = 'super_secret_123'",
  "explanation": "A string literal assigned to a credential-like name...",
  "recommendation": "Store secrets in environment variables..."
}
```


13. Limitations

1. Heuristic rules: false positives possible on legitimate patterns

2. Reg

14. Security and Privacy

- API key stored in .env, excluded from Git via .gitignore

- Sou

15. Future Work

1. Expand rule set to cover C/C++ AST using libclang or tree-sitter

2. Inte

16. Conclusion

AutoCodeGuard demonstrates a practical, modular approach to assisted code review for automotive software development. The combination of deterministic static analysis, LLM advisory review, and TF-IDF guideline retrieval provides an accessible first-pass review tool. The evaluation framework ensures that performance claims are backed by real measurements.

17. References

1. MISRA C:2012 Guidelines ? Motor Industry Software Reliability Association

2. ISO

?? Note: This technical report is a draft. Please verify against the prescribed PCCOE/Tata TechPulse template before final submission.